Django User Authentication & Authorization

1. What is the difference between **authentication** and **authorization** in Django?

- **Authentication** means checking if the user is real (login with username, password).
- **Authorization** means checking what permissions that user has (what pages or actions they can access).

2. Which Django app provides authentication and authorization features by default?

   Django comes with a built-in app called django.contrib.auth that handles both authentication and authorization.

3. What is the role of the AUTH_USER_MODEL setting in Django?

   AUTH_USER_MODEL allows you to specify a custom user model instead of using Django's default User. This is useful if you want to use email instead of username for login or add extra fields to your user model. It ensures Django uses your custom model across the project.

4. How do you access the currently logged-in user in a Django view?

   We can access the current user using request.user in any view.

   ```
   def profile_view(request):
       user = request.user
       return render(request, 'profile.html', {'user': user})
   ```

5. Explain the difference between is_authenticated and is_anonymous properties of a user.

   is_authenticated is True if the user is logged in, while is_anonymous is True if the user is not logged in. They help in conditionally showing content based on login status.

Django User Model & Permissions Management

6. What is the default user model provided by Django, and what fields does it include?

   Django provides the User model by default, which includes fields like username, email, password, first_name, last_name, and flags like is_staff, is_active, and is_superuser.

7. Why and when should you use a **custom user model** in Django?

   We should use a custom user model if you want to change login credentials (e.g., use email instead of username), add extra fields, or customize user behavior. It's best to define a custom model at the start of your project.

8. Write the code to create a custom user model that uses email instead of username for login.

```
from django.contrib.auth.models import AbstractBaseUser, BaseUserManager
from django.db import models
class CustomUserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError("Email is required")
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save()
        return user
    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        return self.create_user(email, password, **extra_fields)
class CustomUser(AbstractBaseUser):
    email = models.EmailField(unique=True)
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    objects = CustomUserManager()
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['first_name', 'last_name']
```

9. How do you assign a group and permissions to a user in Django?

```
from django.contrib.auth.models import Group, Permission, User
user = User.objects.get(username='john')
group = Group.objects.get(name='Editors')
user.groups.add(group)  # Assign group
permission = Permission.objects.get(codename='change_article')
user.user_permissions.add(permission)  # Assign individual permission
```

10. What are the methods has_perm() and has_perms() used for in Django?

has_perm('app_label.permission_codename') checks if a user has a specific permission. has_perms(['perm1', 'perm2']) checks if a user has all permissions in the list. These are useful for controlling access to views or actions.

Implementing Login, Logout, & User Management Features

11. What Django functions are used to handle **login** and **logout**?

authenticate(request, username, password) → verifies credentials.

login(request, user) → logs in the authenticated user.

logout(request) → logs out the current user.

12. Write a simple view to log a user in using authenticate() and login().

```
from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect
def login_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
```

```
            return redirect('home')
        return render(request, 'login.html')
```

13. How do you implement user registration with password hashing in Django?

    Django automatically hashes passwords when you use User.objects.create_user():

    ```
    from django.contrib.auth.models import User
    user = User.objects.create_user(username='john', password='mypassword')
    ```

14. What is the difference between @login_required decorator and LoginRequiredMixin?

    @login_required → decorator for **function-based views**.

    LoginRequiredMixin → mixin for **class-based views**. Both ensure the user is logged in before accessing a view.

15. Write the steps to implement **password reset** functionality in Django.

- Add URL patterns for PasswordResetView, PasswordResetDoneView, PasswordResetConfirmView, and PasswordResetCompleteView.
- Create templates for email, form, and confirmation.
- Configure EMAIL_BACKEND and SMTP settings in settings.py.
- Users can request a password reset link via email and reset their password using the link.